Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 2

Regular Expressions, Tokenization, Edit Distance

Outline

- Regular Expressions
- Words
- Corpora
- Simple Unix Tools for Word Tokenization
- Word and Subword Tokenization
- Word Normalization, Lemmatization and Stemming
- Sentence Segmentation
- Minimum Edit Distance

Basic Text Processing: Regular Expressions

- Regular expressions are used everywhere:
  - Part of every text processing task
    - Not a general NLP solution (for that we use large NLP systems we will see in later lectures)
    - But very useful as part of those systems
      - E.g., searching in text, web scraping, data formatting, text pre-processing.
  - Necessary for data analysis of text data:
    - E.g., pattern-matching
  - A widely used tool in industry and academia

Regular expressions

- A formal language for specifying text strings
- How can we search for mentions of these cute animals in text?

  - woodchuck
  - woodchucks
  - Woodchuck
  - Woodchucks
  - Groundhog
  - groundhogs



Regular Expressions: Disjunctions

- Letters inside square brackets []

Pattern	Matches
[wW]oodchuck	Woodchuck, woodchuck
[1234567890]	Any one digit

- Ranges using the dash `[A-Z]`

Pattern	Matches	
`[A-Z]`	An upper case letter	`Drenched Blossoms`
`[a-z]`	A lower case letter	`my beans were impatient`
`[0-9]`	A single digit	`Chapter 1: Down the Rabbit Hole`

Regular Expressions: Negation in Disjunction

- Caret as first character in [] negates the list
  - Note: Caret means negation only when it's first in []
  - Special characters (., *, +, ?) lose their special meaning inside []

Pattern	Matches	Examples
[^A-Z]	Not an upper case letter	Oyfn pripetchik
[^Ss]	Neither 'S' nor 's'	I have no exquisite reason"
[^.]	Not a period	Our resident Djinn
[e^]	Either e or ^	Look up ^ now

Regular Expressions: Convenient aliases

Pattern	Expansion	Matches	Examples
\d	[0-9]	Any digit	Fahreneit 451
\D	[^0-9]	Any non-digit	Blue Moon
\w	[a-ZA-Z0-9_]	Any alphanumeric or _	Daiyu
\W	[^\w]	Not alphanumeric or _	Look!
\s	[\r\t\n\f]	Whitespace (space, tab)	Look␣up
\S	[^\s]	Not whitespace	Look up

Regular Expressions: More Disjunction

- Groundhog is another name for woodchuck!
- The pipe symbol | for disjunction

Pattern	Matches
groundhog|woodchuck	woodchuck
yours|mine	yours
a|b|c	= [abc]
[gG]roundhog|[Ww]oodchuck	Woodchuck



Wildcards, optionality, repetition: . ? * +

Pattern	Matches	Examples
beg.n	Any char	begin begun beg3n beg n
woodchucks?	Optional s	woodchuck woodchucks
to*	0 or more of previous char	t to too tooo
to+	1 or more of previous char	to too tooo toooo



Stephen C Kleene

Regular Expressions: Anchors

^ $

Pattern	Matches
^[A-Z]	Palo Alto
^[^A-Za-z]	1 "Hello"
\.$	The end.
.$	The end? The end!

The carat matches the start of the line, and the $ matches the end of the line.

A note about Python regular expressions

- In Python regular expressions use backslash "\" for special characters. You must type extra backslashes!
  - "\\d+" to search for 1 or more digits
  - "\n" in Python means the "newline" character, not a "slash" followed by an "n". Need "\\n" for the two characters.
- Instead: use Python's **raw string notation** for regex:
  - r"[tT]he"
  - r"\d+" matches one or more digits
    - instead of "\\d+"

The iterative process of writing regex's

- Find me all instances of the word “the” in a text.

the	Misses capitalized examples
[tT]he	Incorrectly returns “other” or “Theology”
\W[tT]he\W	

False positives and false negatives

The process we just went through was based on fixing two kinds of errors:

1. Not matching things that we should have matched (The)

   False negatives

2. Matching strings that we should not have matched (there, then, other)

   False positives

Characterizing work on NLP

- In NLP we are always dealing with these kinds of errors.
- Reducing the error rate for an application often involves two antagonistic efforts:
  - Increasing coverage (or *recall*) (minimizing false negatives).
  - Increasing accuracy (or *precision*) (minimizing false positives)

Regular expressions play a surprisingly large role

- Widely used in both academia and industry

1. Part of most text processing tasks, even for big neural language model pipelines
    - including text formatting and pre-processing
2. Very useful for data analysis of any text data

Basic Text Processing:
More Regular Expressions – Substitutions and ELIZA

Basic Text Processing: Substitutions

- Substitution in UNIX commands:
  - s/regexp1/pattern/

  change a string matched by a regex to the substitute, another string.

  e.g.: s/colour/color/
- In Python:

```
import re
text = "The colour of the sky is beautiful."
result = re.sub(r'colour', 'color', text)
print(result)
 #Outputs: The color of the sky is beautiful.
```

Capturing Groups

- Say we want to put angles around all numbers:

the 35 boxes → *the <35> boxes*

- Use parentheses () to "capture" a pattern into a numbered register (1, 2, 3…)
- Use \1 to refer to the contents of the register

s/([0-9]+)/<\1>/

- In Python:

```
result2 = re.sub(r'(\d+)',r'<\1>',text2)
```

Capturing groups: multiple registers

- /the (.*)er they (.*), the \1er we \2/

  Matches

- *the faster they ran, the faster we ran*

  But not

- *the faster they ran, the faster we ate*

Question

Exercise:

What if in Python I wanted to surround any first number in a string with "<" and ">" and the second with "[" and "]"?

e.g.

"The 35 boxes and 42 items are nice."

➔

"The <35> boxes and [42] items are nice."

Think about it.

But suppose we don't want to capture?

- Parentheses have a double function: grouping terms, and capturing
- Non-capturing groups: add a ?: after parentheses:

  /(?:some|a few) (people|cats) like some \1/
- matches
  - some cats like some cats
- but not
  - some cats like some people

Lookahead assertions

- (?= pattern) is true if pattern matches, but is **zero-width; doesn't advance character pointer**
- (?! pattern) true if a pattern does not match
- How to match, at the beginning of a line, any single word that doesn’t start with “Volcano”:

  /^(?!Volcano)[A-Za-z]+/

- In Python:

```
# Pattern to match "First" at the beginning of a
line
pattern = r'^First'
# Using re.MULTILINE to allow ^ to match the start
# of each line (e.g. text string on several lines)
matches = re.findall(pattern, text, re.MULTILINE)
```

Simple Application: ELIZA

- Early NLP system that imitated a Rogerian psychotherapist. Developed by Joseph Weizenbaum, 1966.

- Uses pattern matching to match, e.g.,:
  - "I need X"

and translates them into, e.g.

  - "What would it mean to you if you got X?

Simple Application: ELIZA

Men are all alike.

IN WHAT WAY

They're always bugging us about something or other.

CAN YOU THINK OF A SPECIFIC EXAMPLE

Well, my friend made me come here.

YOUR FRIEND MADE YOU COME HERE

He says I'm depressed much of the time.

I AM SORRY TO HEAR YOU ARE DEPRESSED

How ELIZA works

- s/.* I'M (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/
- s/.* I AM (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/
- s/.* all .*/IN WHAT WAY?/
- s/.* always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE?/

Words and Corpora

How many words in a sentence?

- "I do uh main- mainly business data processing"
  - Fragments, filled pauses
- "Khaled's cat in the hat is different from other cats!"
  - **Lemma**: same stem, part of speech, rough word sense
    - cat and cats = same lemma
  - **Word form**: the full inflected surface form
    - cat and cats = different word forms

How many words in a sentence?

they lay back on the Hassi Dalla3ah grass and looked at the stars and their

- **Type**: an element of the vocabulary.
- **Token**: an instance of that type in running text.
- How many?
  - 15 tokens (or 14)
  - 13 types (or 12) (or 11?)

How many words in a corpus?

N = number of tokens

V = vocabulary = set of types, **|*V*|** is size of vocabulary

Heaps Law = Herdan's Law = $|V| = kN^{\beta}$ where often .67 < β < .75 and 10 < k < 100 (empirical parameters)

i.e., the vocabulary size grows with more than the square root of the number of word tokens

Corpus	Tokens = N	Types = |V|
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
Corpus of Contemporary American English	440 million	2 million
Google N-grams	1 trillion	13+ million

Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer (or specific writers),
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.

Corpora vary along dimensions like

- **Language**: 7097 languages in the world
- **Variety**, like African American Language (AAE) varieties.
  - AAE Twitter posts might include forms like "*iont" (I don't)*
- **Code switching**, e.g., Spanish/English, Hindi/English:
  - S/E: Por primera vez veo a @username actually being hateful! It was beautiful:)

  [For the first time I get to see @username actually being hateful! it was beautiful:)]

  - H/E: dost tha or ra- hega ... dont wory ... but dherya rakhe

  ["he was and will remain a friend ... don't worry ... but have faith"]
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Author Demographics**: writer's age, gender, ethnicity, etc.

Corpus datasheets

- Whenever you build a corpus, you should be documenting these decisions in a datasheet for the corpus.

  Motivation:
  - Why was the corpus collected?
  - By whom?
  - Who funded it?

  Situation: In what situation was the text written? (e.g. from work situations like doctors' notes, legal text, parliamentary or congressional proceedings, …)

  Collection process: Data size? If it is a subsample how was it sampled? Was there consent? Pre-processing?

  Annotation process, language variety, demographics, etc.

Text Normalization

- Every NLP task requires text normalization:
  1. Tokenizing (segmenting into) words
  2. Normalizing word formats
  3. Segmenting (into) sentences

Space-based tokenization

- A very simple way to tokenize
  - For languages that use space characters between words
    - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
  - Segment off a token between instances of spaces
  - Additionally, punctuation can be used for this.
- Unix tools for space-based tokenization
  - The "tr" (translate character) command
  - Given a text file, output the word tokens and their frequencies

Simple Tokenization in UNIX

- Given a text file, output the word tokens and their frequencies

Change all non-alpha to newlines

```
tr -sc 'A-Za-z' '\n' < shakes.txt
     | sort
     | uniq -c
```

Sort in alphabetical order

Merge and count each type

```
1945 A
  72 AARON
  19 ABBESS
   5 ABBOT
 ... ...
```

```
25 Aaron
 6 Abate
 1 Abates
 5 Abbess
 6 Abbey
 3 Abbot
... ...
```

```
-s stands for "squeeze
-c stands for complement
```

The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
    # head, displays the first few lines of
    # its input (10 by default).
THE
SONNETS
by
William
Shakespeare
From
fairest
creatures
We
...
```

The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head

A
A
A
A
A
A
A
A
A
A
...
```

More counting (with Normalisation)

- Merging upper and lower case

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z'
'\n' | sort | uniq -c
```

- Sorting the counts

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z'
'\n' | sort | uniq -c | sort -n -r
```

```
   23243 the
   22225 i
   18618 and
   16339 to
   15687 of
   12780 a
   12163 you
   10839 my
   10005 in
    8954  d
```

What happened here?

Issues in Tokenization

- Can't just blindly remove punctuation:
  - m.p.h., Ph.D., AT&T, …
  - prices ($45.55)
  - dates (01/02/06)
  - URLs (http://www.stanford.edu)
  - hashtags (#nlproc)
  - email addresses (someone@cs.colorado.edu)
- Clitic: a word that doesn't stand on its own
  - "are" in we're, French "je" in j'ai, "le" in "l'honneur"
- When should multiword expressions (MWE) be words?
  - Hassi Dalla3ah, rock 'n' roll

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)    # set flag to allow verbose regexps
...     ([A-Z]\.)+        # abbreviations, e.g. U.S.A.
...   | \w+(-\w+)*        # words with optional internal hyphens
...   | \$?\d+(\.\d+)?%?  # currency and percentages, e.g. $12.40, 82%
...   | \.\.\.            # ellipsis
...   | [][.,;"'?():-_`]  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Tokenization in languages without spaces

- Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

- How do we decide where the token boundaries should be?

Word tokenization in Chinese

- Chinese words are composed of characters called "**hanzi**" (or sometimes just "**zi**")
- Each one represents a meaning unit called a **morpheme**.
- Each word has on average 2.4 of them.
- But deciding what counts as a word is complex and not agreed upon.

How to do word tokenization in Chinese?

- 姚明进入总决赛 "Yao Ming reaches the finals"

- 3 words?
  姚明 进入 总决赛
  YaoMing reaches finals

- 5 words?
  姚 明 进入 总 决赛
  Yao Ming reaches overall finals

- 7 characters? (don't use words at all):
  姚 明 进 入 总 决 赛
  Yao Ming enter enter overall decision game

Word tokenization / segmentation

- In Chinese it is common to just treat each character (zi) as a token.
  - So the **segmentation** step is very simple
- In other languages (like Thai and Japanese), more complex word segmentation is required.
  - The standard algorithms are neural sequence models trained by supervised machine learning.

Another option for text tokenization: Byte Pair Encoding

- Instead of
  - white-space segmentation
  - single-character segmentation
- **Use the data** to tell us how to tokenize.
- **Subword tokenization** (because tokens can be parts of words as well as whole words)

Subword tokenization

- Three common algorithms:
  - **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
  - **Unigram language modeling tokenization** (Kudo, 2018)
  - **WordPiece** (Schuster and Nakajima, 2012)
- All have 2 parts:
  - A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
  - A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary.

Byte Pair Encoding (BPE) token learner

Let a vocabulary be the set of all individual characters

= {A, B, C, D,…, a, b, c, d….}

- Repeat:
  - Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
  - Add a new merged symbol 'AB' to the vocabulary
  - Replace every adjacent 'A' 'B' in the corpus with 'AB'.
- Until *k* merges have been done.

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens til k times
  $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
  $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
  $V \leftarrow V + t_{NEW}$ # update the vocabulary
  Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '__' before space in training corpus

Next, separate into letters.

```
def add_end_of_word_symbol(text):
 words = text.split() # Split by spaces
 return ' '.join([word+'__' for word in words])


text = "Hello world! This is an example."
processed_text = add_end_of_word_symbol(text)
print(processed_text)
```

BPE token learner

Original (very fascinating☺) corpus:

low low low low low lowest lowest newer newer newer newer newer newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

corpus	representation
5	l o w _
2	l o w e s t _
6	n e w e r _
3	w i d e r _
2	n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

Merge e r to er

corpus

5 l o w _
2 l o w e s t _
6 n e w er _
3 w i d er _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er _
3 w i d er _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Merge er _ to er_

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

Merge n e to ne

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_, ne

BPE

The next merges are:

Merge	Current Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

BPE token segmenter algorithm

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them
- (test data frequencies don't play a role, just the training data frequencies)

So: merge every e r to er, then merge er _ to er_, etc.

- Result:
  - Test set "n e w e r _" would be tokenized as a full word
  - Test set "l o w e r _" would be two tokens: "low er_"

Properties of BPE tokens

- These tokens usually include frequent words and frequent subwords which are often morphemes like *-est* or *–er*

- A **morpheme** is the smallest meaning-bearing unit of a language.
  - *unlikeliest* has 3 morphemes *un-*, *likely*, and *-est*

Basic Text Processing: Word Normalization & and other issues

- Putting words/tokens in a standard format (normalization)
  - U.S.A. or USA
  - uhhuh or uh-huh
  - Fed or fed
  - am, is, be, are
- This is important, e.g. in Information Extraction and Information Retrieval.

Case folding

- Applications like IR: reduce all letters to lower case
  - Since users tend to use lower case
  - Possible exception: upper case in mid-sentence?
    - e.g., ***General Motors***
    - ***Fed*** vs. ***fed***
    - ***SAIL*** vs. ***sail***
- For sentiment analysis, MT, Information extraction
  - Case is helpful (***US*** versus ***us*** is important)

Lemmatization

Represent all words as their lemma, their shared root

= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*

- Spanish quiero ('I want'), quieres ('you want')

  → querer 'want'

- *He is reading detective stories*

  → *He be read detective story*

Lemmatization: done by Morphological Parsing

- Morphemes:
  - The small meaningful units that make up words
  - **Stems**: The core meaning-bearing units
  - **Affixes**: Parts that adhere to stems, often with grammatical functions
- Morphological Parsers:
  - Parse *cats* into two morphemes *cat* and *s*
  - Parse Spanish *amaren* (if in the future "they would love') into morpheme *amar* 'to love', and the morphological features *3PL* and *future subjunctive*.

Stemming

- Reduce terms to stems, chopping off affixes crudely

This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.



Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note .

Porter Stemmer

- Based on a series of rewrite rules run in series
  - A cascade, in which output of each pass is fed to the next pass
- Some sample rules:

$$\text{ATIONAL} \rightarrow \text{ATE} \quad (\text{e.g., relational} \rightarrow \text{relate})$$

$$\text{ING} \rightarrow \epsilon \quad \text{if stem contains vowel (e.g., motoring} \rightarrow \text{motor)}$$

$$\text{SSES} \rightarrow \text{SS} \quad (\text{e.g., grasses} \rightarrow \text{grass})$$

Dealing with complex morphology is necessary for many languages

- e.g., the Turkish word:

  Uygarlastiramadiklarimizdanmissinizcasina

  `(behaving) as if you are among those whom we could not civilize'

  Uygar `civilized' + las `become'
  + tir `cause' + ama `not able'
  + dik `past' + lar 'plural'
  + imiz 'p1pl' + dan 'abl'
  + mis 'past' + siniz '2pl' + casina 'as if'

Sentence Segmentation

- !, ? mostly unambiguous but **period** "." is very ambiguous
  - Sentence boundary
  - Abbreviations like Inc. or Dr.
  - Numbers like .02% or 4.3
- Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.
  - An abbreviation dictionary can help
- Sentence segmentation can then often be done by rules based on this tokenization.

Minimum Edit Distance

Definition of Minimum Edit Distance

How similar are two strings?

- Spell correction
  - The user typed "graffe"

  Which is closest?
    - graf
    - graft
    - grail
    - giraffe

- Computational Biology
  - Align two sequences of nucleotides

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTTGCCCGAC
```

  - Resulting alignment:

```
-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---
TAG-CTATCAC--GACCGC--GGTCGATTTGCCCGAC
```

- Also for Machine Translation, Information Extraction, Speech Recognition

Edit Distance

- **Minimum edit distance** between two strings: minimum number of editing operations
  - Insertion
  - Deletion
  - Substitution

needed to transform one into the other

Minimum Edit Distance

- Two strings and their **alignment**:

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
```

Minimum Edit Distance

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
d s s   i s
```

- If each operation has a cost of 1
  - Distance between these words is 5
- If substitutions cost 2 (Levenshtein)
  - Distance between them is 8

Alignment in Computational Biology

- Given a sequence of bases

  AGGCTATCACCTGACCTCCAGGCCGATGCCC
  TAGCTATCACGACCGCGGTCGATTTGCCCGAC

- An alignment could be:

  -AGG**CTATCAC**C**T****GACC**T**C**C**A****GG**C**CGA****--TGCCC**---
  T**AG****-CTATCAC****--GACC**G**C****--GG**T**CGA**TT**TGCCC**GAC

- Given two sequences, align each letter to a letter or gap

Other uses of Edit Distance in NLP

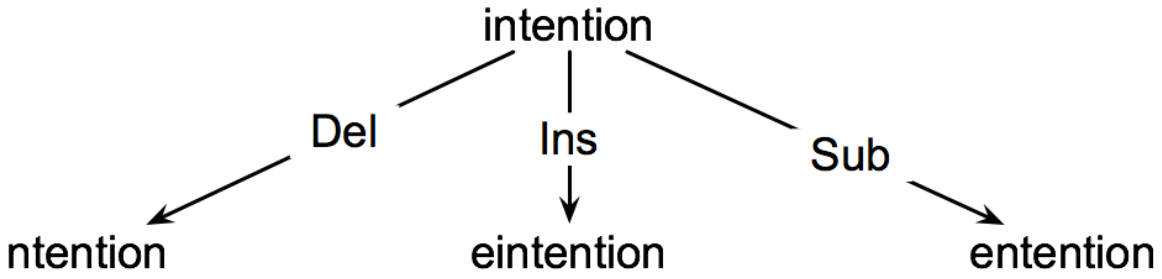
- Evaluating Machine Translation (comparison to reference translations) and speech recognition (compare the output text to a reference transcription)

```
R Spokesman confirms    senior government adviser was shot
H Spokesman said    the senior          adviser was shot dead
              S     I             D                       I
```

- Named Entity Extraction and Entity Coreference
  - IBM Inc. announced today
  - IBM profits
  - Stanford President John Hennessy announced yesterday
  - for Stanford University President John Hennessy

How to find the Min Edit Distance?

- Searching for a path (sequence of edits) from the start string to the final string:
  - **Initial state**: the word we're transforming
  - **Operators**: insert, delete, substitute
  - **Goal state**: the word we're trying to get to
  - **Path cost**: what we want to minimize: the number of edits



Minimum Edit as Search

- But the space of all edit sequences is huge!
  - We can't afford to navigate naïvely
  - Lots of distinct paths wind up at the same state.
    - We don't have to keep track of all of them
    - Just the shortest path to each of those revisited states.

Defining Min Edit Distance

- For two strings
  - X of length *n*
  - Y of length *m*
- We define D(i,j) the edit distance between X[1..i] and Y[1..j] (i.e., the first *i* characters of X and the first *j* characters of Y)
- The edit distance between X and Y is thus D(n,m)

Dynamic Programming for Minimum Edit Distance

- **Dynamic programming**: A tabular computation of D(*n*,*m*)
- Solving problems by combining solutions to subproblems.
- Bottom-up
  - We compute D(i,j) for small *i*,*j*
  - And compute larger D(i,j) based on previously computed smaller values
  - i.e., compute D(*i*,*j*) for all *i* (0 < *i* < n) and *j* (0 < j < m)

Defining Min Edit Distance (Levenshtein)

- **Initialization**

  $D(i,0) = i$

  $D(0,j) = j$

- **Recurrence Relation:**

  For each $i = 1 \ldots M$

  For each $j = 1 \ldots N$

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

- **Termination:**

  $D(N,M)$ is the distance

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

The Edit Distance Table

N	9	8	9	10	11	12	11	10	9	**8**
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

MED: Backtrace for Computing Alignments

- Edit distance isn't sufficient
  - We often need to **align** each character of the two strings to each other
- We do this by keeping a "backtrace"
- Every time we enter a cell, remember where we came from
- When we reach the end,
  - Trace back the path from the upper right corner to read off the alignment

MinEdit with Backtrace

n	9	↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙←↓ 12	↓ 11	↓ 10	↓ 9	↙ **8**
o	8	↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↓ 10	↓ 9	↙ **8**	← 9
i	7	↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	↙ **8**	← 9	← 10
t	6	↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙ **8**	← 9	← 10	←↓ 11
n	5	↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ **8**	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙↓ 10
e	4	↙ 3	← 4	↙← **5**	← **6**	← 7	←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9
t	3	↙←↓ 4	↙←↓ **5**	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙ 7	←↓ 8	↙←↓ 9	↓ 8
n	2	↙←↓ **3**	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↓ 7	↙←↓ 8	↙ 7
i	**1**	↙←↓ 2	↙←↓ 3	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙ 6	← 7	← 8
#	**0**	1	2	3	4	5	6	7	8	9
	#	e	x	e	c	u	t	i	o	n

Adding Backtrace to Minimum Edit Distance

- **Base conditions:** **Termination:**

  `D(i,0) = i` `D(0,j) = j` `D(N,M) is distance`

- **Recurrence Relation:**

```
For each  i = 1…M
      For each  j = 1…N
```

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion} \\ D(i,j-1) + 1 & \text{insertion} \\ D(i-1,j-1) + \begin{cases} 2; \text{ if } X(i) \neq Y(j) & \text{substitution} \\ 0; \text{ if } X(i) = Y(j) \end{cases} \end{cases}$$

$$ptr(i,j) = \begin{cases} \text{LEFT} & \text{insertion} \\ \text{DOWN} & \text{deletion} \\ \text{DIAG} & \text{substitution} \end{cases}$$

Result of Backtrace

- Two strings and their **alignment**:

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
```

Worst Case Performance

- Time: O(n*m)
- Space: O(n*m)
- Backtrace O(n+m)

Weighted Minimum Edit Distance

- Why would we add weights to the computation?
  - Spell Correction: some letters are more likely to be mistyped than others
  - Biology: certain kinds of deletions or insertions are more likely than others

Confusion matrix for spelling errors

sub[X, Y] = Substitution of X (incorrect) for Y (correct)

X \ Y (correct)	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1	342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	9	0	1	0	5	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	8	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	0	3	0	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	0	5	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	2	3	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	0	14	0	2	4	14	39	0	0	0	18	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	2	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	2	43	0	0	4	0	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0


~ `
! 1
@ 2
3
$ 4
% 5
^ 6
& 7
* 8
(9
) 0
— -
+ =
Tab
Q
W
E
R
T
Y
U
I
O
P
{ [
}]
Caps Lock
A
S
D
F
G
H
J
K
L
: ;
" '
Enter
Shift
Z
X
C
V
B
N
M
< ,
> .
? /
Shift
Ctrl
Alt
Alt

Weighted Min Edit Distance

- **Initialization:**

$$D(0,0) = 0$$

$$D(i,0) = D(i-1,0) + del[x(i)]; \quad 1 < I \leq N$$

$$D(0,j) = D(0,j-1) + ins[y(j)]; \quad 1 < j \leq M$$

- **Recurrence Relation:**

$$D(i,j) = \min \begin{cases} D(i-1,j) + del[x(i)] \\ D(i,j-1) + ins[y(j)] \\ D(i-1,j-1) + sub[x(i),y(j)] \end{cases}$$

- **Termination:**

$$D(N,M) \text{ is distance}$$

Where did the name, dynamic programming, come from?

…The 1950s were not good years for mathematical research. [the] Secretary of Defense …had a pathological fear and hatred of the word, research…

I decided therefore to use the word, "**programming**".

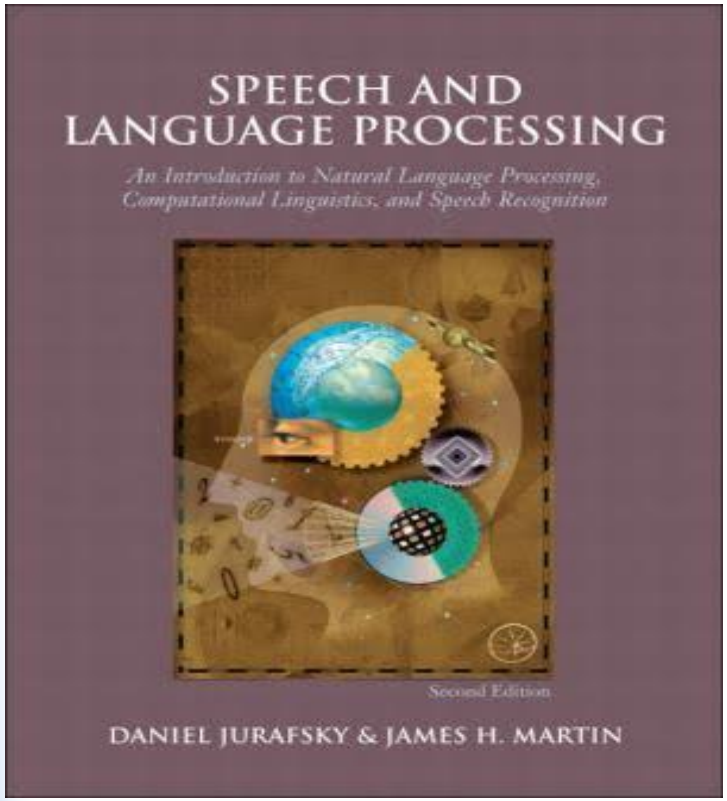
I wanted to get across the idea that this was dynamic, this was multistage… I thought, let's … take a word that has an absolutely precise meaning, namely **dynamic**… it's impossible to use the word, **dynamic**, in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It's impossible.

Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to."

Richard Bellman, "Eye of the Hurricane: an autobiography" 1984.

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.

This chapter: